Digital Signal Processing – Assignment 1
Fourier Transform: Voice Analysis, Enhancement and Aural Exciter

William Chambers (GUID: 2906463C)
Saad Aziz (GUID: 2917444A)

University of Glasgow, School of Engineering

October 19, 2026

1 Recordings

Will spoke one English sentence, "*Saad and Will are electrowizards*", recorded at 5 cm and 1 m as uncompressed mono WAV files (48 kHz, 24-bit). The full spectrum up to 20 kHz (the human audible spectrum) is available, and neither recording is clipped (peaks at −22.5 dBFS and −44.5 dBFS).

2 Task 1: Time and Frequency Analysis

The samples are normalised by 2^{31}, the full scale of the 32-bit signed integers used to store the 24-bit audio. The spectrum is $X = \frac{1}{N}\,\mathrm{FFT}(x)$, plotted as $20\log_{10}|X|$ in decibels relative to full scale (dBFS): the loudest level possible in a digital system is 0 dBFS. Only the positive half is shown, because the spectrum of a real signal is mirrored.

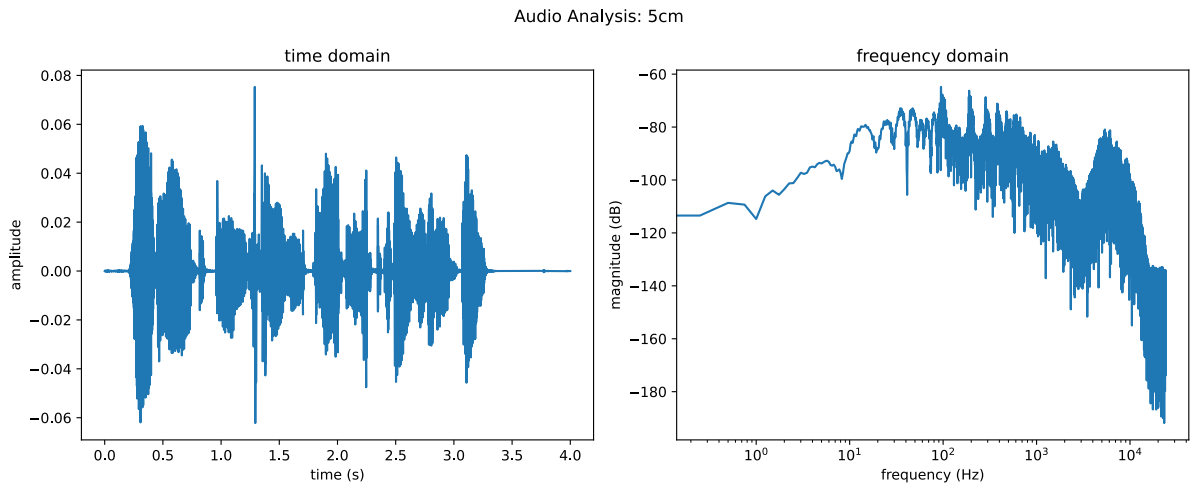


Figure 1: Voice recorded at 5 cm: (left) time domain, (right) frequency domain.

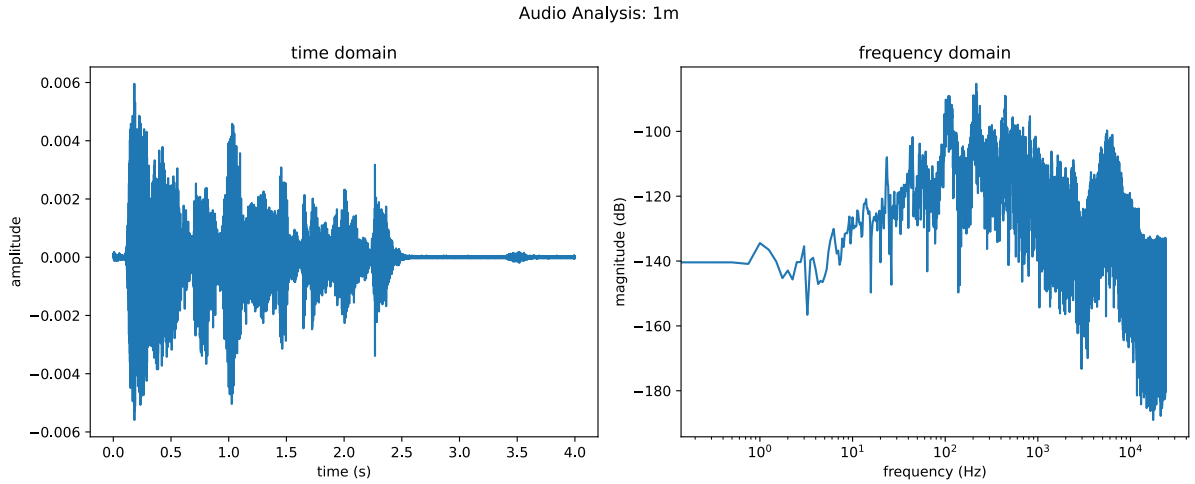


Figure 2: Voice recorded at 1 m: (left) time domain, (right) frequency domain.

Fundamentals. In both recordings the vowels appear as series of evenly spaced peaks. The lowest peak of each series was taken as the fundamental, because the peaks above it sit at integer multiples of it (e.g. 95, 190, 285 and 380 Hz at 5 cm), which identifies a harmonic series rather than noise. The fundamentals lie between about 94 and 103 Hz at 5 cm and 101 and 112 Hz at 1 m, as expected for an adult male voice. Series with slightly different fundamentals within these ranges belong to different vowels. They vary between recordings because these are separate takes. Peaks below about 80 Hz have no harmonic series and were not chosen; these are low-frequency rumble from pop energy and room noise.

Consonants. Above about 2 kHz the harmonic series fades and a broad hump without harmonic structure appears, peaking at 5 kHz to 6 kHz: the energy of consonants such as /s/ and /z/. It is present on the frequency spectrum up to about 14.5 kHz at 5 cm but only 12.5 kHz at 1 m, where the weaker consonant energy is lost in the noise floor sooner.

Noise. Above 16 kHz at 5 cm and 14 kHz at 1 m, the spectrum is flat; between the end of the consonant range and this floor (14.5 kHz to 16 kHz and 12.5 kHz to 14 kHz) the spectrum falls steeply as the consonant energy fades. The level is the same in both recordings even though the voice is about 20 dB quieter at 1 m, so it cannot come from the voice: it is most likely electronic noise.

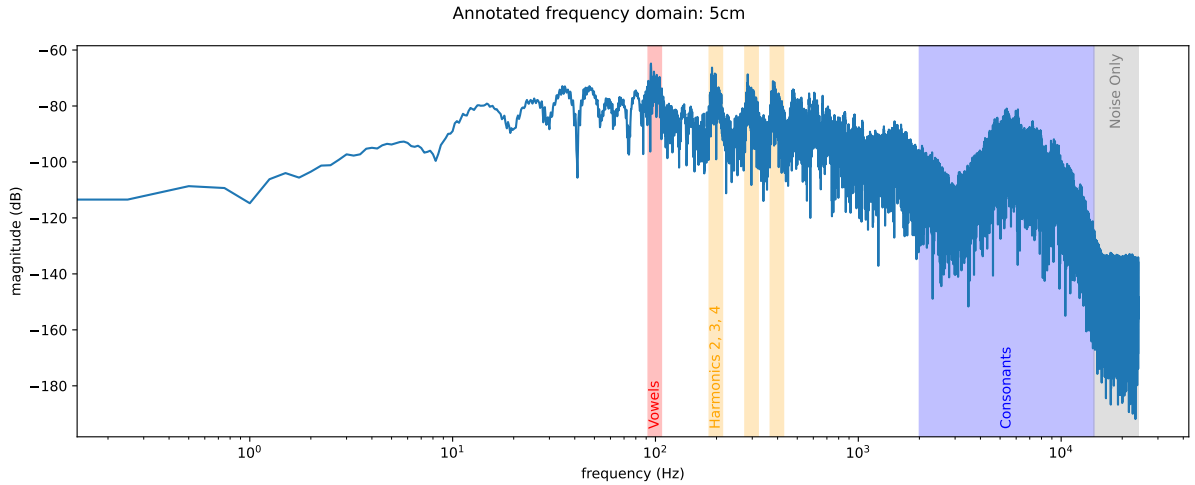


Figure 3: Annotated spectrum at 5 cm: vowel fundamentals, harmonics, consonant range and noise-only region.

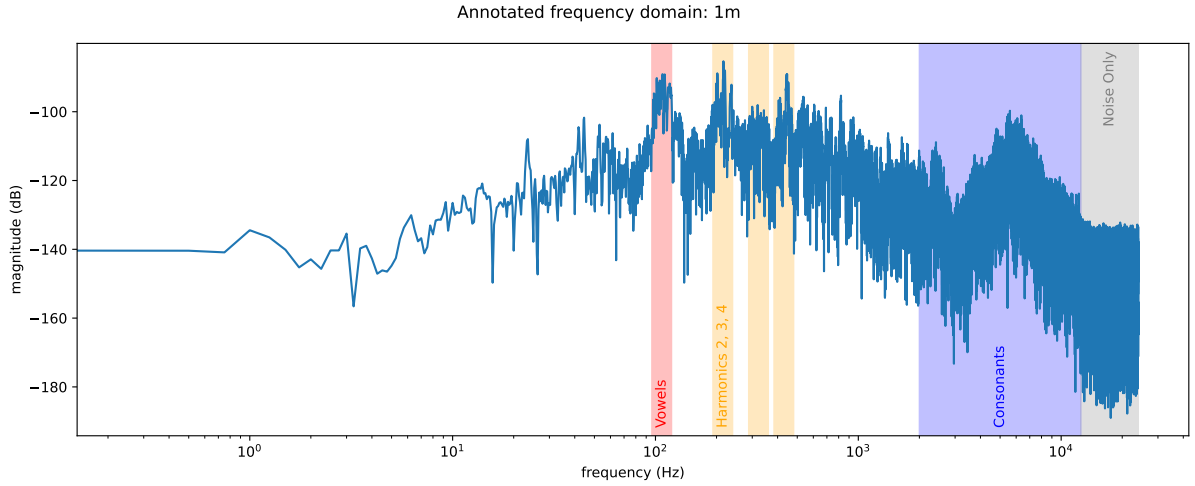
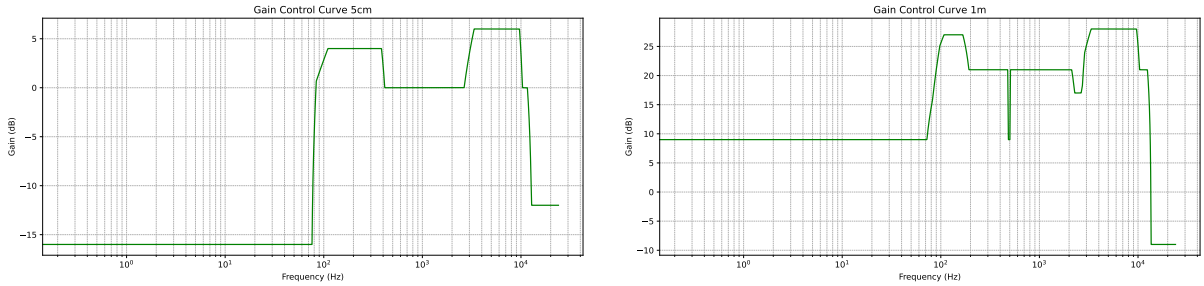


Figure 4: Annotated spectrum at 1 m: vowel fundamentals, harmonics, consonant range and noise-only region.

3 Task 2: Speech Enhancement

Method. Each recording is transformed with the FFT, then every frequency bin is multiplied by a gain array (an array of ones), and the result is transformed back with the IFFT. `gain_ctrl(low, high, dB, q)` adds one band to the array: a trapezoid over the selected frequencies, converted to linear gain with $10^{\mathrm{dB}/20}$ and mirrored onto the negative frequencies so that the IFFT output is real (`np.real` only removes rounding residue). Hard band edges caused audible ringing (Gibbs phenomenon), so each edge ramps over a fraction q of the band width, with the nominal edge at the middle of the ramp. An identity test (all-ones gain array) confirmed that the pipeline alone does not change the signal at all. The outputs are scaled by $2^{31} - 1$ and saved as 32-bit integer WAV.



(a) 5 cm **(b)** 1 m

Figure 5: Gain control curves applied to each recording (net gain per frequency, including the 21 dB level match at 1 m).

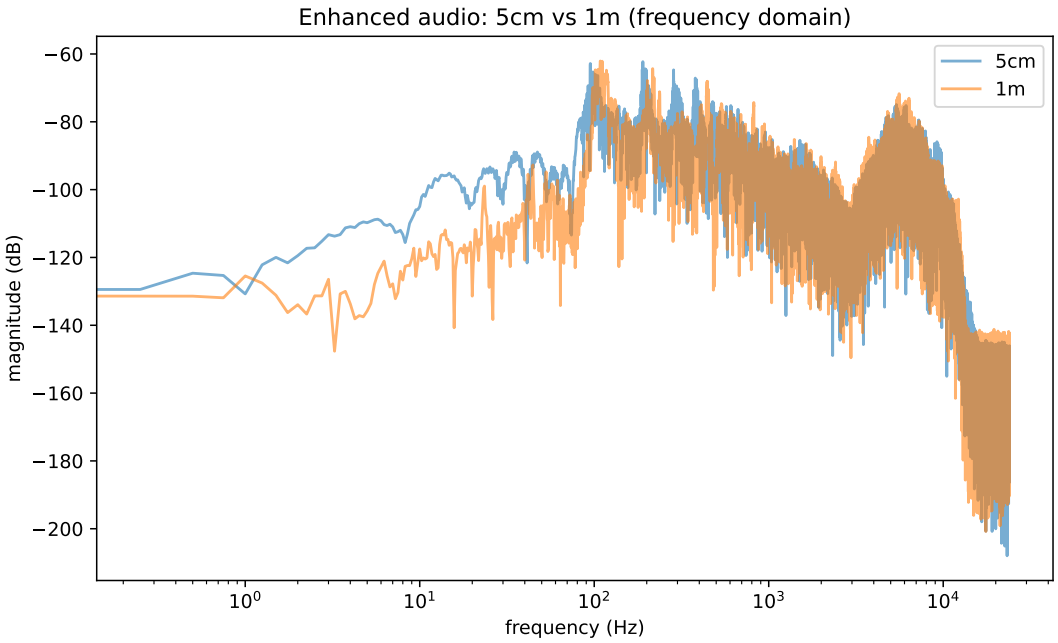


Figure 6: Enhanced spectra, 5 cm vs 1 m.

5 cm. A low cut of −16 dB below 80 Hz removes rumble and part of the pop energy (the pop at 1.28 s to 1.30 s lies mostly below 150 Hz). It stops at 80 Hz to keep the ≈95 Hz fundamental, so the pop is slightly reduced but not removed. The low end is already strong at this proximity, so the bass boost is small: 4 dB between 95 Hz to 400 Hz; this range covers the first few harmonics for depth. The consonant region (3 kHz to 10 kHz) is boosted by 6 dB for clarity and 'brightness'. A high cut of −12 dB above 12.2 kHz reduces hiss. It is kept moderate because consonant energy extends to 14.5 kHz at this distance.

Table 1: EQ Settings for 5 cm Recording

Band (Hz)	Gain (dB)	q	Purpose
0–80	−16	0.1	Low cut
95–400	+4	0.1	Bass boost
3000–10000	+6	0.1	Brightness
12200–Nyq	−12	0.1	High cut

1 m. The whole spectrum is raised by 21 dB to match the volume of both recordings. A low cut of −12 dB below 85 Hz removes room rumble. Bass is lost with distance, so the gain is larger to compensate: 6 dB between 95 Hz to 180 Hz, around the fundamental. The low cut and bass boost edges meet at 85 Hz to 95 Hz and initially rang with $q = 0.1$, so both use $q = 0.3$, which removed the ringing at about 95 Hz. A −12 dB notch at 480 Hz to 505 Hz removes a 500 Hz tone visible in the noise floor after the voice ends. A −4 dB cut at 2.2 kHz to 2.8 kHz reduces the room resonance around 2.5 kHz. The consonant region is boosted by 7 dB, slightly more than at 5 cm, because the distant recording is duller. A −30 dB high cut above 13 kHz, just above where the consonants end, removes the hiss raised by the level match. There was no pop at this distance; however, since the volume was increased so much, the SNR was reduced to the point that the signal sounds noisy.

Table 2: EQ Settings for 1 m Recording

Band (Hz)	Gain (dB)	q	Purpose
0–Nyq	+21	0	Volume match
0–85	−12	0.3	Low cut
95–180	+6	0.3	Bass boost
480–505	−12	0.25	Notch cut
2200–2800	−4	0.25	Room resonance cut
3000–10000	+7	0.1	Brightness
13000–Nyq	−30	0.1	High cut

4 Task 3: Aural Exciter

Design. The exciter follows the Aphex sidechain (Figure 7), ignoring the Big Bottom sidechain, and runs on the raw 5 cm recording. A band-pass filter is created by zeroing all FFT bins outside a specified region (two `gain_ctrl` cuts). Unlike Task 2, these edges are not ramped ($q = 0$): a sharp cut keeps neighbouring voice content out of the non-linearity and passes only the chosen products, where the ringing is barely audible (see V3). A band-pass was used instead of the tunable high-pass, so that less of the voice enters the non-linearity and fewer unwanted harmonics are created. The band is normalised to a peak of 1, so that the drive does not depend on the recording level, and passed through $\tanh(\text{drive} \cdot x)$. This is almost linear for small inputs and compresses progressively as they grow. Because tanh is an odd function, it creates only odd-order harmonics. A second band-pass keeps the wanted products further up the spectrum, which are scaled by the mix value and added to the dry signal. The pop was not removed before processing; however, since it lies mostly below the sidechain band, the pop is barely amplified.

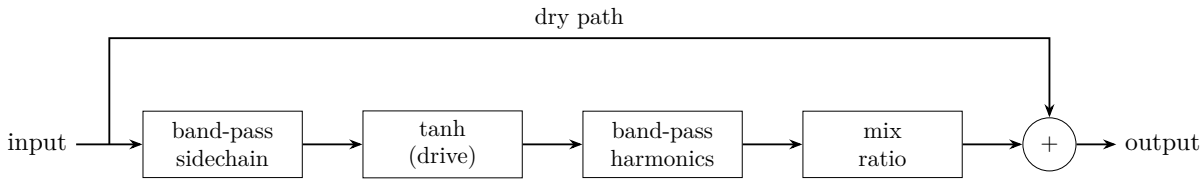


Figure 7: Block diagram of the implemented exciter.

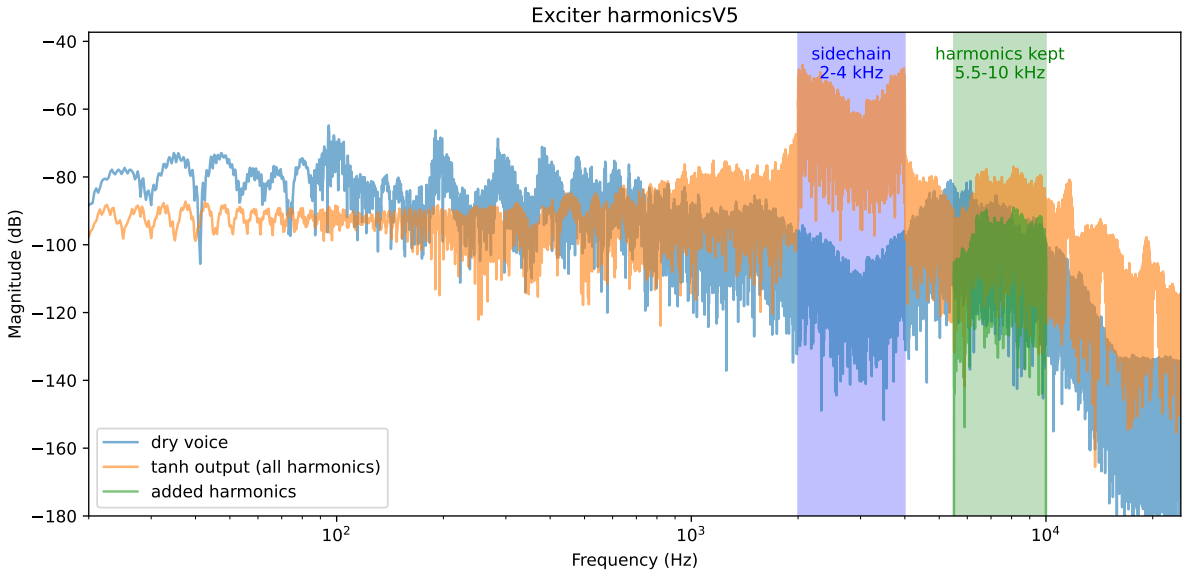


Figure 8: Final design: dry voice, full tanh output and the added harmonics. The tanh output contains only odd harmonics of the sidechain band; it is shown before band selection and mix, so it is not to scale with the voice.

Iterations. The versions are summarised in Table 3.

Table 3: Exciter versions (dry recording).

Version	Sidechain (Hz)	Keep band (Hz)	Drive	Mix
V1	1750–2250	5000–7000	10	0.5
V2	1750–2250	5000–7000	3	0.1
V3	2000–4000	4000–12000	3	0.25
V4	2000–4000	5500–12000	6	0.15
V5 (final)	2000–4000	5500–10000	4	0.25

V1: sidechain 1750 Hz to 2250 Hz, centred on 2 kHz so that its 3rd harmonic lands at 6 kHz, where the ear is most sensitive; keep band 5 kHz to 7 kHz; drive 10, mix 0.5. The plot confirms that only odd harmonics appear (6, 10 and 14 kHz), but the voice sounds robotic and metallic, and the output peak is very large, so the drive and mix were reduced.

V2: drive 3, mix 0.1. The peak is unchanged and the "k" and click sounds are clearer, but the effect is confined to one narrow region, so the bands were widened.

V3: sidechain 2 kHz to 4 kHz (3rd harmonics now span 6 kHz to 12 kHz), keep band 4 kHz to 12 kHz, mix 0.25. The effect is broader but rings, and the peak rises to 1.7 times the dry peak.

V4: the keep band starts at 5.5 kHz, just below the lowest 3rd harmonic, and ends at 12 kHz; mix 0.15; drive raised in steps to 6. The effect is clearly audible, but the consonants become too prominent (a lisp effect).

V5 (final): drive back to 4, keep band ending at 10 kHz to remove the content behind the lisp, and the mix tuned by ear to 0.25. With the narrower keep band, the ringing was no longer audible.

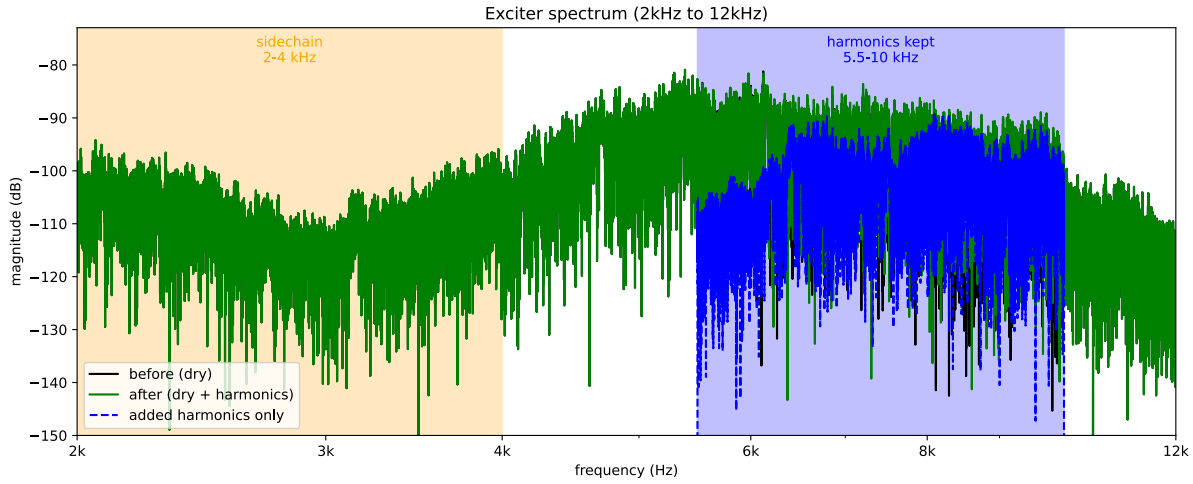


Figure 9: Final design, 2 kHz to 12 kHz: spectrum before (dry) and after the exciter, and the added harmonics only, with the sidechain band (2 kHz to 4 kHz) and the harmonics kept (5.5 kHz to 10 kHz) marked.

Final design. Sidechain 2 kHz to 4 kHz, drive 4, keep band 5.5 kHz to 10 kHz, mix 0.25. The keep band holds the 3rd harmonics of 2 kHz to 3.3 kHz and excludes both the sidechain band itself and the content above 10 kHz. The output volume in the FFT is roughly the same and the peak rises by approximatley 4 dB.

The k and s sounds are more pronounced and the voice sounds closer, while its level barely changes, which is the intended effect of an exciter.

A Acknowledgement of GenAI Use

Tools: Claude Opus 5.5 (Anthropic); Gemini 3.1 Pro (Google); GitHub Copilot (Microsoft).
Use: Claude Opus 5.5 was used to debug code syntax and logic and to suggest ways of making the code more efficient; to check the Task 1 readings against our own analysis; to explain the exciter and how a non-linearity creates harmonics; to fix a gain curve that rose at both ends of the spectrum; and to plot the overlay of both enhanced recordings and the harmonics for Task 3. Gemini 3.1 Pro was used for debugging, building LaTEX framework for the report, helping plot graphs, and checking the report. Gemini AI overviews for general lookups, and GitHub Copilot for inline autocomplete.

B Usage instructions

From a terminal, while in the project folder, run:

`python3 run_tasks.py`

The scripts need `numpy`, `scipy` and `matplotlib`. The plots of Tasks 1–3 are opened with `plt.show()`. The files `enhanced_5cm.wav`, `enhanced_1m.wav` and `exciter_5cm.wav` are written to the working directory. `run_tasks.py` imports `task1_plot.py`, `task2_enhance.py` and `task3_exciter.py`, which must be in the same folder.

C Python Code

C.1 run_tasks.py

```python
import os
import matplotlib.pyplot as plt
from scipy.io import wavfile
import numpy as np

os.chdir(os.path.dirname(os.path.abspath(__file__))) # run from any folder: .wav inputs and outputs are next to this file

# Assignmnet files
from task1_plot import plot_audio
from task2_enhance import gain_ctrl, enhancer, plot_gain_array, plot_overlay
from task3_exciter import exciter, plot_harmonics

# ----- Load audio ----- files

# Recommended: 48kHz + Signed 24-bit PCM + Mono (imperative)
sample_rate_5cm, audio_5cm = wavfile.read("wav/5cm.wav")
audio_5cm = audio_5cm / (2**31)  # NORMALISE (32bits to store 24bit audio)

sample_rate_1m, audio_1m = wavfile.read("1m.wav")
audio_1m = audio_1m / (2**31)


# ------ TASK 1 ------

# Plot audio files
plot_audio("5cm", sample_rate_5cm, audio_5cm)
plot_audio("1m", sample_rate_1m, audio_1m)

# ------ TASK 2 ------

#array of n ones (floats). Each value is a multiplier for the corresponding bin
```

```
gain_array_5cm = np.ones(len(audio_5cm))
gain_array_1m = np.ones(len(audio_1m))

# --- ENHANCER TEST ---
#does input match output with no changes applied ?
#test_5cm = np.ones(len(audio_5cm)) #untouched gain arrays
#test_1m = np.ones(len(audio_1m))
#print("Identity test 5cm:", np.allclose(enhancer(audio_5cm, test_5cm), audio_5cm))
#print("Identity test 1m:", np.allclose(enhancer(audio_1m, test_1m), audio_1m))
# --- END OF TEST ---

# --- 5cm ---
#gain curve - gain_ctrl(Low freq, High freq, Gain change, ..., ..., q=0)
gain_ctrl(0, 80, -16, sample_rate_5cm, gain_array_5cm, q=0.1) #LOW CUT: Remove rumble and some POP energy
gain_ctrl(95, 400, 4, sample_rate_5cm, gain_array_5cm, q=0.1) #BASS BOOST: The first couple harmonics for depth
gain_ctrl(3000, 10000, 6, sample_rate_5cm, gain_array_5cm, q=0.1) #BRIGHTNESS: The noisy consonant region + extra for clarity
gain_ctrl(12200, sample_rate_5cm //2 - 1, -12, sample_rate_5cm, gain_array_5cm, q=0.1) #HIGH CUT : Remove any hiss

#apply gain + save
plot_gain_array(gain_array_5cm,sample_rate_5cm,"5cm")
enhanced_5cm = enhancer(audio_5cm, gain_array_5cm)
wavfile.write("enhanced_5cm.wav", sample_rate_5cm, ((2**31-1)*enhanced_5cm).astype(np.int32))

# --- 1m ---
#gain curve
gain_ctrl(0, sample_rate_1m//2 - 1, 21, sample_rate_1m, gain_array_1m) #VOLUME: Match to 5cm recording (raw RMS is 21dB quieter)
gain_ctrl(0, 85, -12, sample_rate_1m, gain_array_1m, q=0.3) #LOW CUT: trim room rumble (less 'boxy')
gain_ctrl(95, 180, 6, sample_rate_1m, gain_array_1m, q=0.3)  #BASS BOOST: fundamental higher @1m + restoring depth to voice
gain_ctrl(480, 505, -12, sample_rate_1m, gain_array_1m, q=0.25)  #Reduce the ~500Hz whine in the noise floor
gain_ctrl(2200, 2800, -4, sample_rate_1m, gain_array_1m, q=0.25)  #ROOM RESONANCE: 1m bump ~2.5 kHz vs 5cm
gain_ctrl(3000, 10000, 7, sample_rate_1m, gain_array_1m, q=0.1) #BRIGHTNESS
gain_ctrl(13000, sample_rate_1m//2 - 1, -30, sample_rate_1m, gain_array_1m, q=0.1) #HIGH CUT consonants end lower

#apply gain + save
plot_gain_array(gain_array_1m,sample_rate_1m,"1m")
enhanced_1m = enhancer(audio_1m, gain_array_1m)
wavfile.write("enhanced_1m.wav", sample_rate_1m, ((2**31-1)*enhanced_1m).astype(np.int32)) # int32 scaled up (mention in report)

# Generate overlay comparison plot
#plot_overlay([("5cm", sample_rate_5cm, enhanced_5cm), ("1m", sample_rate_1m, enhanced_1m)])

# ------ TASK 3 ------

# call exiter
exciter_5cm, nonlin_5cm, harmonics_5cm = exciter(audio_5cm, sample_rate_5cm, 2000, 4000, 5500, 10000, drive=4, mix=0.25)

# plot annotated
#plot_harmonics(audio_5cm, nonlin_5cm, harmonics_5cm, sample_rate_5cm, 2000, 4000, 5500, 10000, "V5")

# apply exciter + save
```

```
wavfile.write("exciter_5cm.wav", sample_rate_5cm, ((2**31-1)*exciter_5cm).astype(np.int32))

# Show audio plots
plt.show()
```

C.2 task1_plot.py

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

# Function to plot audio data in time and frequency domain
def plot_audio(name, sample_rate, audio):
    plt.figure(figsize=(12, 5))

    # Time domain
    n = len(audio)
    time = np.linspace(0, n / sample_rate, num=n)  # array of times

    # Frequency domain
    audio_fft = (1/n) * fft(audio)

    # decibels relative to full scale (dBFS)
    mag_fft = 20 * np.log10(np.abs(audio_fft))
    freqs = (np.arange(n // 2) * sample_rate / n)  # frequency bins for plotting (//2 returns
    integer not float)

    print(f"File: {name}, Sample Rate: {sample_rate}")

    # time domain plot
    plt.suptitle(f"Audio Analysis: {name}")
    plt.subplot(1, 2, 1)
    plt.title("time domain")
    plt.plot(time, audio)
    plt.xlabel("time (s)")
    plt.ylabel("amplitude")

    # frequency domain plot
    plt.subplot(1, 2, 2)
    plt.title("frequency domain")
    plt.plot(freqs, mag_fft[: n // 2])
    plt.xlabel("frequency (Hz)")
    plt.xscale("log")  # make logarithmic scale like standard EQ's/visualisers
    plt.ylabel("magnitude (dB)")

    plt.tight_layout()  # stop axes overlapping
```

C.3 task2_enhance.py

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft, ifft


# GAIN CONTROL
#function that describes gain adjustments to specific frequency ranges.
def gain_ctrl(low, high, amp_dB, sample_rate, gain_array, q=0):
    """
    q = 0    -> ramps one FFT bin wide (default) (rectangular)
    q = 0.25 -> each edge ramps over 25% of the bandwidth (trapezoidal)
    q = 1    -> ramps meet in the middle (triangular)
```

```
    """
    n = len(gain_array)
    half = n // 2
    bin_hz = sample_rate / n
    freqs = np.arange(half + 1) * bin_hz

    q = np.clip(q, 0.0, 1.0)                  # >1 would make the ramps cross
    ramp_hz = max(q * (high - low), bin_hz)   # never narrower than one bin
    r = ramp_hz / 2

    # bands that start at DC or end at Nyquist: push that ramp outside the spectrum so the ends get the full gain
    if low <= 0:
        low = -ramp_hz
    if high >= sample_rate // 2 - 1:
        high = sample_rate / 2 + ramp_hz

    amp_lin = 10.0 ** (amp_dB/ 20.0)
    amp_points = [low - r, low + r, high - r, high + r]
    freq_points = [   1.0, amp_lin,  amp_lin,      1.0]
    profile_lin = np.interp(freqs, amp_points, freq_points, left=1.0, right=1.0)

    gain_array[: half + 1] *= profile_lin
    gain_array[half + 1:] = gain_array[1:n - half][::-1]   # mirror to the negative frequencies

    return gain_array

# PLOT GAIN
def plot_gain_array(gain_array, sample_rate, name):
    n = len(gain_array)
    freqs = (np.arange(n // 2) * sample_rate / n)  # frequency bins for plotting

    plt.figure(figsize=(10, 5))
    plt.title("Gain Control Curve " + name)
    plt.plot(freqs, 20 * np.log10(np.abs(gain_array[:n // 2])), color='green')
    plt.xscale("log")
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Gain (dB)")
    plt.grid(which='both', linestyle='--', linewidth=0.5)
    plt.tight_layout()  # plt.show() is called once at the end of run_tasks.py

# --- ENHANCER ---
#takes gain array and applies it to audio
def enhancer(audio, gain_array):

    #FFT the full signal (all (n) bins)
    audio_fft = fft(audio)

    #multiply the bins by gain_array
    audio_enhanced = audio_fft * gain_array

    #IFFT gives back all (n) samples, then keep the real part
    audio_ifft = ifft(audio_enhanced)
    audio_ifft_real = np.real(audio_ifft) #extract the real part (IM part should be ~0 since gain array is mirrored)

    return audio_ifft_real

# Audio overlay plot
def plot_overlay(signals):
    # signals: list of (label, sample_rate, audio)
    plt.figure(figsize=(8, 5))

```

```
    for label, fs, audio in signals:
        n = len(audio)
        mag_fft = 20 * np.log10(np.abs((1/n) * np.fft.fft(audio)) + 1e-12)  # +1e-12 avoids log
    (0)
        freqs = np.arange(n // 2) * fs / n

        plt.plot(freqs, mag_fft[: n // 2], alpha=0.6, label=label)

    plt.title("Enhanced audio: 5cm vs 1m (frequency domain)")
    plt.xlabel("frequency (Hz)")
    plt.xscale("log")
    plt.ylabel("magnitude (dB)")
    plt.legend()

    plt.tight_layout()
```

C.4 task3_exciter.py

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

from task2_enhance import gain_ctrl, enhancer


#Band selection (used for both the sidechain and the harmonic selection)
def band_pass(audio, sample_rate, lo, hi):
    gain_array = np.ones(len(audio))
    gain_ctrl(0, lo, -np.inf, sample_rate, gain_array) #low cut (0-lo)
    gain_ctrl(hi, sample_rate//2 -1, -np.inf, sample_rate, gain_array) #high cut (hi+)

    return enhancer(audio, gain_array)

def exciter(audio, sample_rate, in_lo, in_hi, keep_lo, keep_hi, drive, mix):
    #isolate the band
    sidechain_exciter = band_pass(audio, sample_rate, in_lo, in_hi)
    sidechain_exciter = sidechain_exciter / np.max(np.abs(sidechain_exciter)) #peak of 1, so
    drive is always effective

    #introduce non-linearity
    nonlin = np.tanh(drive * sidechain_exciter)

    #isolate the band around 6kHZ - this rejetcs 5th harmonics since they lie outside the region
    target_band = band_pass(nonlin, sample_rate, keep_lo, keep_hi)

    #MIXER
    harmonics = mix * target_band

    return audio + harmonics, nonlin, harmonics #output, all harmonics from tanh, harmonics
    actually added

#Plot the harmonics on their own (for report)
def plot_harmonics(audio, nonlin, harmonics, sample_rate, in_lo, in_hi, keep_lo, keep_hi, name):

    n = len(audio)
    freqs = np.arange(n // 2) * sample_rate / n  # frequency bins for plotting

    plt.figure(figsize=(10, 5))
    plt.title("Exciter harmonics" + name)

    for label, x in (("dry voice", audio),
                     ("tanh output (all harmonics)", nonlin),
                     ("added harmonics", harmonics)):
```

```
        mag = np.abs(fft(x)[:n // 2]) / n
        mag_db = 20 * np.log10(mag + 1e-12)  # +1e-12 avoids log(0)
        plt.plot(freqs[1:], mag_db[1:], alpha=0.6, label=label)  # skip DC for log axis

    plt.xscale("log")
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude (dB)")
    plt.xlim(20, sample_rate / 2)
    plt.ylim(bottom=-180)

    # Get axis limits after plotting so annotations sit near the top
    y_bottom, y_top = plt.ylim()
    pad = 0.03 * (y_top - y_bottom)
    x_mid = lambda lo, hi: np.sqrt(lo * hi)  # geometric mean = centre on log axis

    # Annotate sidechain band
    plt.axvspan(in_lo, in_hi, color="blue", alpha=0.25)
    plt.text(x_mid(in_lo, in_hi), y_top - pad,
             f"sidechain\n{in_lo/1000:g}-{in_hi/1000:g} kHz",
             color="blue", ha="center", va="top")

    # Annotate harmonics kept
    plt.axvspan(keep_lo, keep_hi, color="green", alpha=0.25)
    plt.text(x_mid(keep_lo, keep_hi), y_top - pad,
             f"harmonics kept\n{keep_lo/1000:g}-{keep_hi/1000:g} kHz",
             color="green", ha="center", va="top")

    plt.legend(loc="lower left")
    plt.tight_layout()
```